INSTITUTE
OF LAW,
POLITICS AND
DEVELOPMENT



Sant'Anna
School of Advanced Studies – Pisa

Data Analysis in R using Agentic AI Assistants

Session 1: Getting set up

Prof. Kevin Koehler
kevin.koehler@santannapisa.it

Section 1

1. Welcome

1.1 The class

Aims:

1. Introduce you to R, the language most quantitative political science is done in, and to RStudio, the program you use it in

1.1 The class

Aims:

1. Introduce you to R, the language most quantitative political science is done in, and to RStudio, the program you use it in
2. Carry out the basic tasks of data analysis: managing data, visualizing it, and running regressions

1.1 The class

Aims:

1. Introduce you to R, the language most quantitative political science is done in, and to RStudio, the program you use it in
2. Carry out the basic tasks of data analysis: managing data, visualizing it, and running regressions
3. Reproduce two published political science studies

1.1 The class

Aims:

1. Introduce you to R, the language most quantitative political science is done in, and to RStudio, the program you use it in
2. Carry out the basic tasks of data analysis: managing data, visualizing it, and running regressions
3. Reproduce two published political science studies
4. Do all of this with an **AI coding assistant** — and learn to tell when it is right and when it is wrong

1.2 Schedule

1.	Getting set up	Fri 23 Oct, 16–18
2.	Data management	Fri 30 Oct, 16–18
3.	Visualization	Mon 2 Nov, 9–11
4.	R programming	Mon 9 Nov, 16–18
5.	Exploring data	Mon 16 Nov, 16–18
6.	Linear regression	Mon 23 Nov, 16–18
7.	Replication I: US soft power	Mon 30 Nov, 9–11
8.	Regression for yes/no outcomes	Mon 7 Dec, 16–18
9.	Replication II: coups and counterbalancing	Mon 14 Dec, 16–18

Textbook: Kennedy & Waggoner (2021), *Introduction to R for Social Scientists*. It predates AI assistants: we use it for the R, and do the AI part in class.

1.3 Why R?

- It is **free**, and it runs on any computer.

1.3 Why R?

- It is **free**, and it runs on any computer.
- It is what most quantitative political science is written in. Replication files for published articles usually come in R or Stata.

1.3 Why R?

- It is **free**, and it runs on any computer.
- It is what most quantitative political science is written in. Replication files for published articles usually come in R or Stata.
- It does far more than statistics: graphics, maps, web scraping, reports, even these slides.

1.4 What has changed

- Until recently, a course like this spent weeks on **syntax**: where the brackets go, what a function is called, why the code throws an error.

1.4 What has changed

- Until recently, a course like this spent weeks on **syntax**: where the brackets go, what a function is called, why the code throws an error.
- Today an AI assistant writes working R code faster than anyone can look it up. Typing code is no longer the hard part.

1.4 What has changed

- Until recently, a course like this spent weeks on **syntax**: where the brackets go, what a function is called, why the code throws an error.
- Today an AI assistant writes working R code faster than anyone can look it up. Typing code is no longer the hard part.
- So this course spends that time on the part that is still hard.

1.5 What has not changed

The assistant's code **runs**. Whether it is **right** is a different question:

- Did merging two datasets quietly duplicate some rows?

None of these produce an error message. **Catching them is your job** — and it is what this course teaches.

1.5 What has not changed

The assistant's code **runs**. Whether it is **right** is a different question:

- Did merging two datasets quietly duplicate some rows?
- Did a code for "don't know" end up inside an average?

None of these produce an error message. **Catching them is your job** — and it is what this course teaches.

1.5 What has not changed

The assistant's code **runs**. Whether it is **right** is a different question:

- Did merging two datasets quietly duplicate some rows?
- Did a code for "don't know" end up inside an average?
- Does the result really support the sentence written about it?

None of these produce an error message. **Catching them is your job** — and it is what this course teaches.

1.6 What you will leave with

You will be able to:

- tell an assistant clearly what you want
- read the code it produces
- check the result against something independent
- explain and defend every step of an analysis

You will *not*:

- write R from a blank page without help
- become a programmer

That is fine. Few researchers write code from scratch any more. All of them still need to know whether their numbers are right.

Section 2

2. The working rule

2.1 The working rule

You may have the assistant write any code you like.

You must be able to **explain every line** you submit and **justify every choice** in it.

"The AI did it" is not an answer to "why did you drop these respondents?"

2.2 How you pass

- The course is **pass/fail**.

2.2 How you pass

- The course is **pass/fail**.
- Final deliverable: a **code file** (an R script or Quarto document) with a short analysis of your choice. Due on the last day of class, **Monday 14 December**.

2.2 How you pass

- The course is **pass/fail**.
- Final deliverable: a **code file** (an R script or Quarto document) with a short analysis of your choice. Due on the last day of class, **Monday 14 December**.
- You then **discuss it with me**. I will pick two or three lines and ask: *What does this do? How did you check that it is right?*

2.2 How you pass

- The course is **pass/fail**.
- Final deliverable: a **code file** (an R script or Quarto document) with a short analysis of your choice. Due on the last day of class, **Monday 14 December**.
- You then **discuss it with me**. I will pick two or three lines and ask: *What does this do? How did you check that it is right?*
- If you can answer, you pass. The working rule is the exam.

2.3 Keep a log

In every project, keep a file called `log.md`. After each session, add one line:

```
30 Oct | model: [name] | it said it removed missing
                         values;
                        nrow() showed nothing was removed
```

- What the assistant got wrong (or claimed to do, but didn't)

Two minutes a week. By December it is your best preparation for the final discussion.

2.3 Keep a log

In every project, keep a file called `log.md`. After each session, add one line:

```
30 Oct | model: [name] | it said it removed missing
                         values;
                        nrow() showed nothing was removed
```

- What the assistant got wrong (or claimed to do, but didn't)
- How you noticed

Two minutes a week. By December it is your best preparation for the final discussion.

2.3 Keep a log

In every project, keep a file called `log.md`. After each session, add one line:

```
30 Oct | model: [name] | it said it removed missing
                         values;
                        nrow() showed nothing was removed
```

- What the assistant got wrong (or claimed to do, but didn't)
- How you noticed
- Which model you were using

Two minutes a week. By December it is your best preparation for the final discussion.

Section 3

3. Getting set up

3.1 Three tools

R is the **language**. It does the actual computing.

RStudio is the **interface**: a window where you write code and see data, results and plots.

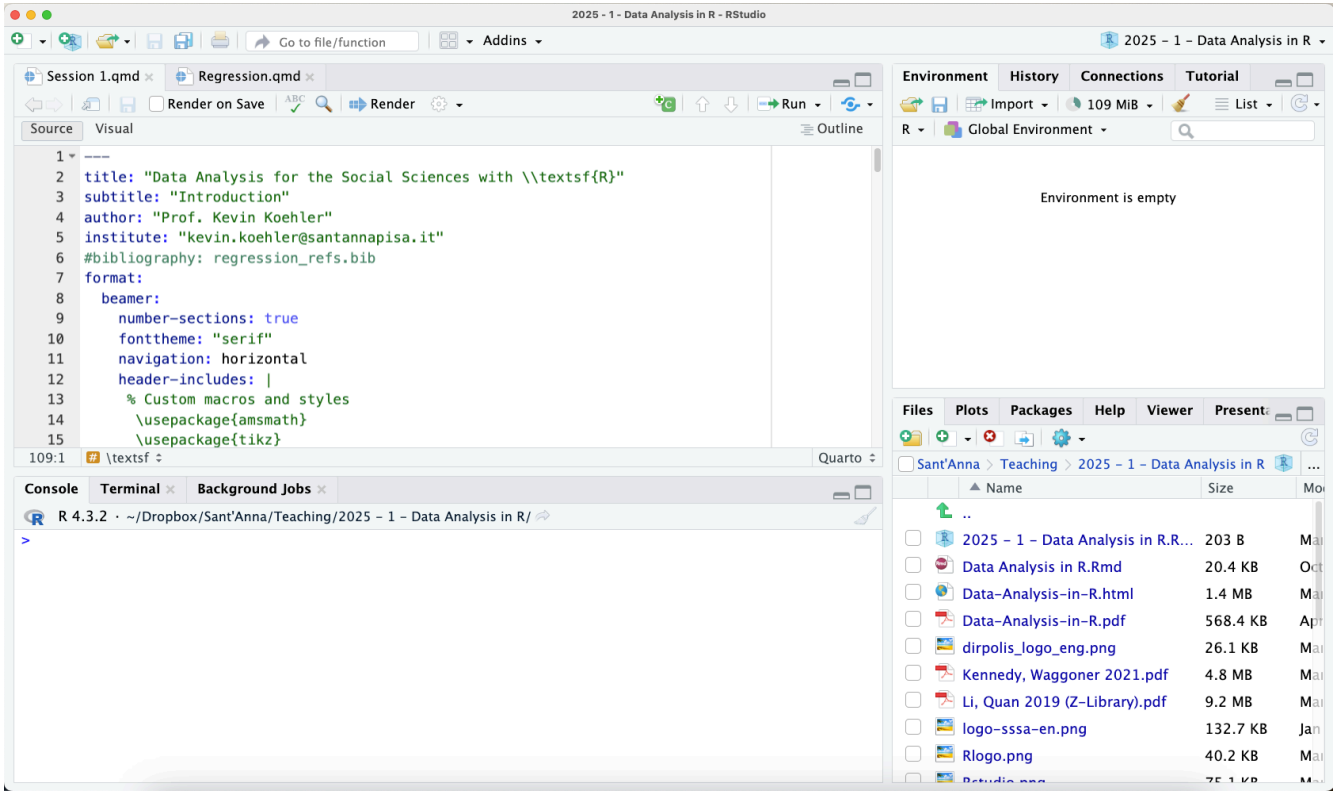
OpenCode is the **assistant**. Unlike a chat window, it can read the files in your project, write new ones and run commands on your computer.



1. R: `cran.r-project.org`
2. RStudio Desktop: `posit.co/downloads`
3. OpenCode: `opencode.ai/download`

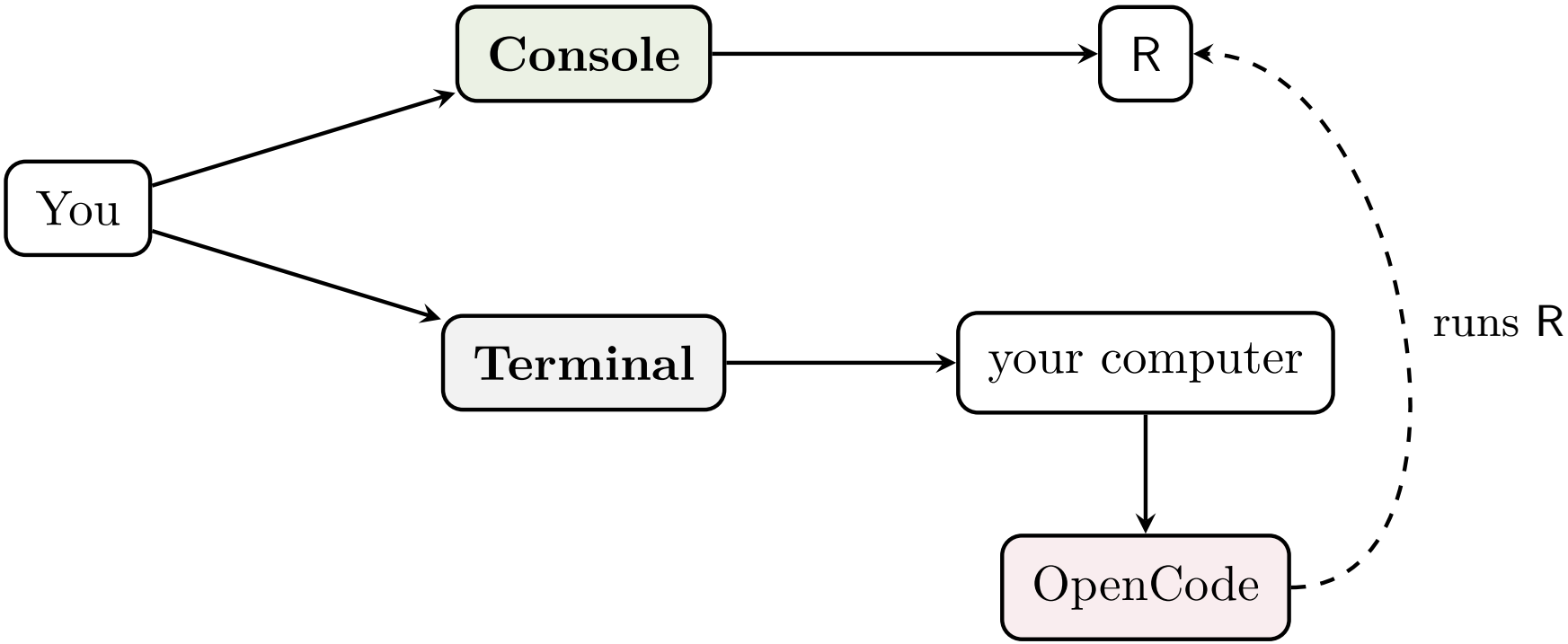
Install R first, then RStudio.

3.2 The RStudio window



Top left: your **scripts**. Bottom left: the **Console**. Top right: the **Environment** (your data). Bottom right: files, plots and help.

3.3 Console or Terminal?



Both are tabs at the bottom left of RStudio. The **Console** talks to R. The **Terminal** talks to your computer. OpenCode lives in the Terminal (or in its own app window) and runs R through your computer.

3.4 Step 1: a project

A **project** keeps everything for one piece of work in one folder, and tells R where to find your files.

1. Unzip the course folder somewhere you will find it again

```
getwd()
```

3.4 Step 1: a project

A **project** keeps everything for one piece of work in one folder, and tells R where to find your files.

1. Unzip the course folder somewhere you will find it again
2. In RStudio: `File > New Project > Existing Directory`

```
getwd()
```

3.4 Step 1: a project

A **project** keeps everything for one piece of work in one folder, and tells R where to find your files.

1. Unzip the course folder somewhere you will find it again
2. In RStudio: `File > New Project > Existing Directory`
3. Choose the course folder and create the project

```
getwd()
```

3.4 Step 1: a project

A **project** keeps everything for one piece of work in one folder, and tells R where to find your files.

1. Unzip the course folder somewhere you will find it again
2. In RStudio: `File > New Project > Existing Directory`
3. Choose the course folder and create the project
4. Check where R is looking — type this in the Console:

```
getwd()
```

3.5 What is in the course folder

`data/`	the Tunisia survey (`tunisia_survey.csv`) and its codebook. Never change these files.
`scripts/`	empty: your R code goes here
`output/`	empty: charts and tables go here
`AGENTS.md`	standing instructions for the assistant (more after the break)
`opencode.json`	tells OpenCode to **ask before** it changes a file or runs a command
`log.md`	your log: one line for each thing you asked the assistant and what you checked

3.6 Step 2: start OpenCode

1. Start OpenCode **in the project folder**: either open the folder in the OpenCode app, or open the **Terminal** tab in RStudio and type `opencode`.

Important: if you use the app, open the **same folder** as your RStudio project. Otherwise the two are looking at different files.

3.6 Step 2: start OpenCode

1. Start OpenCode **in the project folder**: either open the folder in the OpenCode app, or open the **Terminal** tab in RStudio and type `opencode`.
2. Connect MiniMax with the course key (setup guide on the course GitHub), then choose the course model: **MiniMax M2**. In the terminal: `/connect`, then `/models`.

Important: if you use the app, open the **same folder** as your RStudio project. Otherwise the two are looking at different files.

3.6 Step 2: start OpenCode

1. Start OpenCode **in the project folder**: either open the folder in the OpenCode app, or open the **Terminal** tab in RStudio and type `opencode`.
2. Connect MiniMax with the course key (setup guide on the course GitHub), then choose the course model: **MiniMax M2**. In the terminal: `/connect`, then `/models`.
3. No key yet, or it does not work? Use a free OpenCode Zen model for today.

Important: if you use the app, open the **same folder** as your RStudio project. Otherwise the two are looking at different files.

3.7 Step 3: test it

Type these to OpenCode, one at a time:

To OpenCode

List the files in this project and say what each folder is for.

To OpenCode

Run Rscript -e "sessionInfo()" and tell me which version of R you found.

OpenCode should **ask your permission** before running the second one. If it does not, it was probably started in the wrong folder.

3.8 Checkpoint

Before the break, everyone should have:

1. R and RStudio opening without errors

Finished? Help your neighbour. Stuck after five minutes? Work with your neighbour today and we fix your installation after class.

3.8 Checkpoint

Before the break, everyone should have:

1. R and RStudio opening without errors
2. The course project open, with `getwd()` pointing at it

Finished? Help your neighbour. Stuck after five minutes? Work with your neighbour today and we fix your installation after class.

3.8 Checkpoint

Before the break, everyone should have:

1. R and RStudio opening without errors
2. The course project open, with `getwd()` pointing at it
3. OpenCode running in the project, with the course model selected

Finished? Help your neighbour. Stuck after five minutes? Work with your neighbour today and we fix your installation after class.

3.8 Checkpoint

Before the break, everyone should have:

1. R and RStudio opening without errors
2. The course project open, with `getwd()` pointing at it
3. OpenCode running in the project, with the course model selected
4. OpenCode having listed your files and found R

Finished? Help your neighbour. Stuck after five minutes? Work with your neighbour today and we fix your installation after class.

Section 4

4. What an assistant actually does

4.1 A chat window and an assistant

	Chat window	**OpenCode**
Sees your data?	only what you paste in	reads the files in your project
Writes code?	as text you copy	directly into files
Runs code?	no	yes, on your computer
Checks its own work?	no	no

The last row is the one that matters. An assistant that can act is more useful — and its mistakes travel further.

4.2 You stay in control

When OpenCode wants to write a file or run a command, it stops and asks. You can answer:

once | always | reject

- In this course: choose **once**, and read what it is about to do before you approve it.

4.2 You stay in control

When OpenCode wants to write a file or run a command, it stops and asks. You can answer:

once | always | reject

- In this course: choose **once**, and read what it is about to do before you approve it.
- It only asks because our `opencode.json` tells it to. Out of the box, it would just go ahead.

4.2 You stay in control

When OpenCode wants to write a file or run a command, it stops and asks. You can answer:

once | always | reject

- In this course: choose **once**, and read what it is about to do before you approve it.
- It only asks because our `opencode.json` tells it to. Out of the box, it would just go ahead.
- Clicking "yes" without reading is the same as not having the question.

4.3 Demo

To OpenCode

Read the file data/tunisia_survey.csv and the codebook. Write a script scripts/01_first_look.R that loads the data with the tidyverse, reports the number of respondents, and makes a bar chart of the variable july25. Run it.

At each step, ask:

- It wants to write a file or run a command. **Should we let it?**

4.3 Demo

To OpenCode

Read the file data/tunisia_survey.csv and the codebook. Write a script scripts/01_first_look.R that loads the data with the tidyverse, reports the number of respondents, and makes a bar chart of the variable july25. Run it.

At each step, ask:

- It wants to write a file or run a command. **Should we let it?**
- It wrote a script. **What is in it?** We open it and read it, line by line.

4.3 Demo

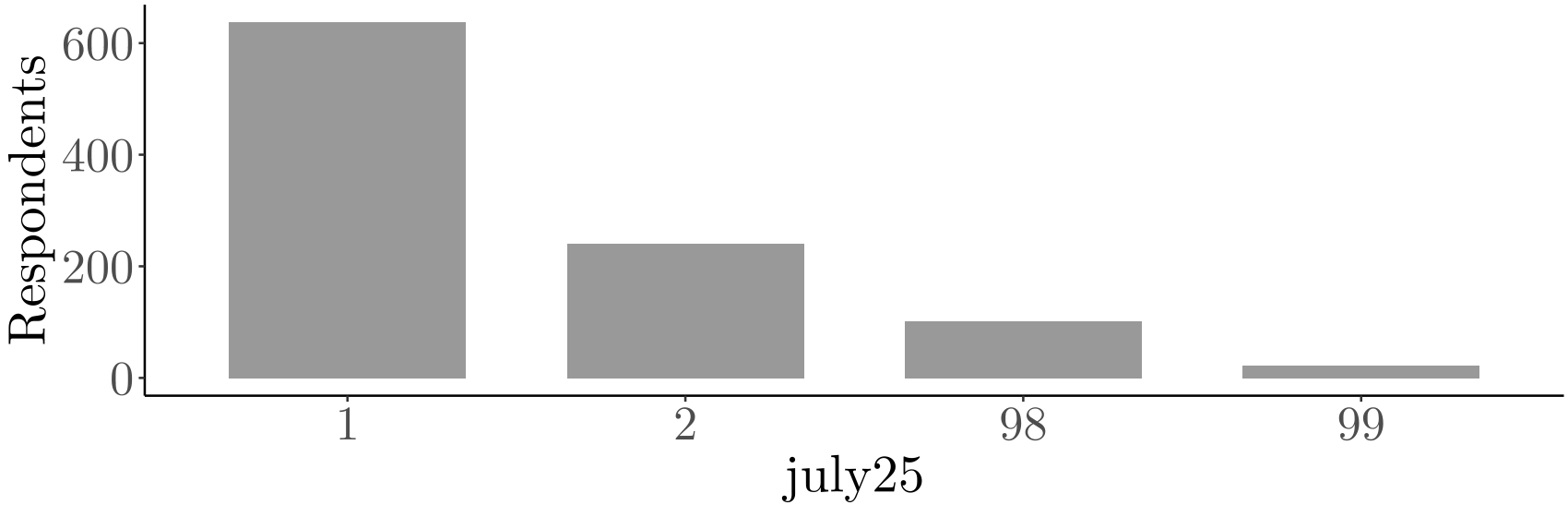
To OpenCode

Read the file data/tunisia_survey.csv and the codebook. Write a script scripts/01_first_look.R that loads the data with the tidyverse, reports the number of respondents, and makes a bar chart of the variable july25. Run it.

At each step, ask:

- It wants to write a file or run a command. **Should we let it?**
- It wrote a script. **What is in it?** We open it and read it, line by line.
- It ran the script and produced a chart. **Is the chart right?**

4.4 The chart looks fine. Does it?



Codebook: 1 = "a necessary correction", 2 = "undermines the transition", **98 = don't know, 99 = declined**. What are the two bars on the right? The "average answer" comes out at 13.2 — on a question answered 1 or 2. More in Session 5.

4.5 Standing instructions: `AGENTS.md`

OpenCode reads `AGENTS.md` at the start of every session, whichever model you use. Ours says, among other things:

- use the `tidyverse`, and comment every step in plain language

You will add your own rules as the course goes on. Do not type `/init`: it replaces our file with one the assistant writes itself.

4.5 Standing instructions: `AGENTS.md`

OpenCode reads `AGENTS.md` at the start of every session, whichever model you use. Ours says, among other things:

- use the `tidyverse`, and comment every step in plain language
- never change the files in `data/`

You will add your own rules as the course goes on. Do not type `/init`: it replaces our file with one the assistant writes itself.

4.5 Standing instructions: `AGENTS.md`

OpenCode reads `AGENTS.md` at the start of every session, whichever model you use. Ours says, among other things:

- use the `tidyverse`, and comment every step in plain language
- never change the files in `data/`
- after every step that can change the number of rows, report how many are left

You will add your own rules as the course goes on. Do not type `/init`: it replaces our file with one the assistant writes itself.

4.5 Standing instructions: `AGENTS.md`

OpenCode reads `AGENTS.md` at the start of every session, whichever model you use. Ours says, among other things:

- use the `tidyverse`, and comment every step in plain language
- never change the files in `data/`
- after every step that can change the number of rows, report how many are left
- never drop missing values without saying how many

You will add your own rules as the course goes on. Do not type `/init`: it replaces our file with one the assistant writes itself.

4.6 Plan first, then build

Press `Tab` to switch between two modes:

Plan

OpenCode reads your files and **proposes** what it would do. It changes nothing.

Build

OpenCode **acts**: it writes files and runs commands (after asking).

For anything more than a small task: ask for a plan, read it, correct it, *then* switch to Build. Fixing a plan is much cheaper than fixing code.

4.7 Free models and your data

- The free models are free partly because the providers may **keep what you send them**.

4.7 Free models and your data

- The free models are free partly because the providers may **keep what you send them**.
- The data in this course are public, so that is fine.

4.7 Free models and your data

- The free models are free partly because the providers may **keep what you send them**.
- The data in this course are public, so that is fine.
- Do **not** point a free model at confidential data of your own: interviews for your thesis, unpublished survey data, anything you promised to keep private.

Section 5

5. Reading R

5.1 You will read more R than you write

```r
ages <- c(23, 70, 41, 35)  # store numbers
mean(ages)                 # apply a function
```

```
[1] 42.25
```

- `<-` **stores** something under a name (an *object*). Read it as "gets".

5.1 You will read more R than you write

```r
ages <- c(23, 70, 41, 35)  # store numbers
mean(ages)                 # apply a function
```

```
[1] 42.25
```

- `<-` **stores** something under a name (an *object*). Read it as "gets".
- `c()` **combines** values into a vector.

5.1 You will read more R than you write

```
ages <- c(23, 70, 41, 35)  # store numbers
mean(ages)                 # apply a function
```

```
[1] 42.25
```

- `<-` **stores** something under a name (an *object*). Read it as "gets".
- `c()` **combines** values into a vector.
- `mean()` is a **function**: a word followed by brackets, with the input inside.

5.1 You will read more R than you write

```
ages <- c(23, 70, 41, 35)  # store numbers
mean(ages)                 # apply a function
```

```
[1] 42.25
```

- `<-` **stores** something under a name (an *object*). Read it as "gets".
- `c()` **combines** values into a vector.
- `mean()` is a **function**: a word followed by brackets, with the input inside.
- Everything after `#` is a **comment**: a note for humans, ignored by R.

5.2 Data frames

A **data frame** is a table: each column is a variable, each row an observation (here: a respondent).

```
survey <- data.frame(
  age    = c(23, 70, 41),
  region = c("Sfax", "Kebili", "Tunis"))
survey$age                          # one column
```

```
[1] 23 70 41
```

```
survey$region[survey$age > 30]   # who is over 30?
```

```
[1] "Kebili" "Tunis"
```

`$` picks a column. `[ ]` picks rows. `>` and `==` *ask* a question; `<-` *stores* an answer.

5.3 Packages and help

Functions live in **packages**. R comes with the basics; we add the `tidyverse`:

```
install.packages("tidyverse")  # once
library(tidyverse)             # every session
```

To find out what a function does, ask R, not the assistant:

```
?mean
```

The help page is written by the people who wrote the function. It is the independent check.

Section 6

6. Your turn



Sant'Anna
School of Advanced Studies – Pisa

6.1 Exercise: build a project, then check it

To OpenCode

Set up this project for a small analysis of the Tunisia survey: create sensible folders if any are missing, a README explaining the structure, and a script that loads and cleans the data. Explain each file you create.

Check — without asking the assistant:

1. Open every file it created. Does the README describe what is actually there?

2. Pick one line of the cleaning script you do not understand. Look up the function with `?` in RStudio. Write down what it does in your own words.

3. Did the "cleaning" change the data? How many rows before and after? Check with `nrow()` yourself.

4. Add one line to `log.md`.

Section 7

7. Wrapping up

7.1 What did you find?

- Did the assistant's explanation match the files it created?

The lesson for the whole course: **the assistant's account of its own work is something to check**, just like its code.

7.1 What did you find?

- Did the assistant's explanation match the files it created?
- Did "cleaning" do what it said it did?

The lesson for the whole course: **the assistant's account of its own work is something to check**, just like its code.

7.1 What did you find?

- Did the assistant's explanation match the files it created?
- Did "cleaning" do what it said it did?
- Who found something the assistant described confidently — and got wrong?

The lesson for the whole course: **the assistant's account of its own work is something to check**, just like its code.

7.2 For next time

Read:

- Kennedy & Waggoner (2021), Introduction and Chapter 2 — if you have not yet
- Kennedy & Waggoner (2021), Chapter 3

Do:

- Keep your project folder: we build on it next week
- Make sure R, RStudio and OpenCode all work on your laptop
- Add your first line to `log.md`

Session 2 (Friday 30 October): data management — selecting, filtering, recoding and merging, and the ways an assistant gets them quietly wrong.